SoftwareOrbits

Full-Stack Engineering Assessment

Real-Time Cinema Seat Reservation System

Estimated Time: 4–5 hours | **Level:** Mid / Senior | **Stack:** TypeScript · Node.js · Next.js

Objective

Build a **real-time cinema seat reservation system** where multiple users can reserve seats for a movie show.

The primary goal of this assessment is **not** to build a beautiful UI, but to design and implement a system that remains **correct under high concurrency**.

> The application must guarantee that **a seat can never be reserved by more than one user**, even when many users attempt to reserve the same seats simultaneously.

Additionally, the system should be designed with **horizontal scalability** in mind. Assume the backend will run on **multiple server instances**, and your solution should maintain correctness and consistency across all instances.

You are free to make architectural decisions and choose the technologies you believe are most appropriate. You are encouraged to use AI-assisted coding tools.

Tech Stack

Required

- TypeScript
- Next.js (Frontend)

Backend — choose one of the following:

- Express
- Fastify
- NestJS
- Any other Node.js framework

Database — entirely your choice. Examples:

- PostgreSQL
- MySQL
- MongoDB
- SQLite
- Any other database you believe fits the problem

Scenario

A cinema has a single movie showing with **50 seats**. Users can reserve one or more available seats.

The application is **real-time**: when one user reserves a seat, every connected user should immediately see the updated seat availability without refreshing the page. The implementation of real-time communication is your choice. Examples include:

- WebSockets
- Socket.IO
- Server-Sent Events (SSE)
- Any other suitable solution

> Assume the application is deployed behind a load balancer and the backend is running on **multiple instances**. Your design should ensure that reservation correctness and real-time updates continue to work in this environment.

Functional Requirements

Backend

Implement APIs (or equivalent) to:

- Retrieve all seats
- Retrieve seat availability
- Reserve one or multiple seats
- Return meaningful errors when seats are no longer available

Each reservation should include:

- Reservation ID
- User ID
- Reserved seats
- Timestamp

Third-Party Booking API

In addition to the frontend-facing APIs, expose a separate API intended for **third-party booking partners**. This API should:

- Allow external systems to reserve one or more seats
- Enforce the exact same business rules and concurrency guarantees as the frontend
- Prevent double-booking even when reservations originate simultaneously from both the frontend and third-party integrations
- Return appropriate success and error responses

> The expectation is that **all reservation paths share the same underlying booking logic**, rather than maintaining separate implementations.

Frontend

Build a simple interface where users can:

- View all seats
- Clearly distinguish available and reserved seats
- Select one or more seats
- Submit a reservation
- See loading and error states
- Receive real-time updates when another user reserves seats
- Reflect seat availability immediately without requiring a page refresh

UI polish is **not** the focus. Correctness and usability are.

High-Concurrency Simulation (Core Requirement)

Your application must include a mechanism to simulate high traffic. This can be implemented as an API endpoint, a script, a button in the UI, a CLI command, or any other reasonable approach.

When triggered, the simulation should create:

- **100 concurrent users**
- Attempting to reserve seats simultaneously
- From the same pool of available seats

The simulation should also be capable of generating requests from **both the frontend reservation API and the third-party booking API**, demonstrating that the system remains consistent regardless of the request source.

Example:

```
Available Seats
A1
A2
A3
A4
A5
```

The simulation starts. 100 concurrent reservation requests are fired. Many users intentionally attempt to reserve the same seats. Your implementation must guarantee that:

- A seat is never reserved twice
- Database consistency is maintained
- Successful reservations are valid
- Failed reservations return appropriate responses
- Connected clients receive real-time updates reflecting the latest seat availability
- Correctness is maintained even when the application is running on multiple backend instances

The implementation details are entirely up to you.

What We Are Evaluating

We're interested in your engineering decisions. Specifically:

- Project architecture
- Real-time communication

- Backend design
- Database schema
- Concurrency handling
- Transaction management
- API design
- Horizontal scalability across multiple server instances
- Error handling
- Frontend state management
- TypeScript quality
- Testing strategy

Deliverables

Please include:

- Complete source code
- README containing:
  - Setup instructions
  - Assumptions
  - Architectural decisions
  - Trade-offs
  - Explanation of how the solution maintains correctness when deployed across multiple instances
  - Explanation of how the third-party booking API shares reservation logic with the primary application
- Any scripts required to run the concurrency simulation

Bonus (Optional)

If time permits, implement one or more of the following:

- Reservation expiration (e.g. automatically release seats after 5 minutes)
- Reservation cancellation
- Retry mechanism
- Integration or unit tests covering concurrent reservations
- Authentication
- Optimistic UI updates
- Docker setup

Notes

There is **no single correct solution**. We expect you to make reasonable engineering decisions and explain any trade-offs where appropriate.

> The assessment is intentionally designed so that AI-generated code alone is insufficient. We are evaluating how you approach a realistic engineering problem involving concurrency, consistency, real-time communication, distributed system design, and API architecture.

Good luck — we look forward to reviewing your work.